

Appendix

Code snippets

MainApp.java

RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

```
package com.retailpos.main;

import com.retailpos.model.Product;
import com.retailpos.model.Customer;
import com.retailpos.model.Bill;
import com.retailpos.service.InventoryService;
import com.retailpos.service.BillingService;
import com.retailpos.service.AuthenticationService;
import com.retailpos.util.FileUtil;
import com.retailpos.util.ValidationUtil;
import java.util.Scanner;
import java.util.ArrayList;
import java.io.IOException;

public class MainApp {

    private static Scanner scanner = new Scanner(System.in);
    private static InventoryService inventoryService;
    private static BillingService billingService;
    private static AuthenticationService authenticationService;
    private static String currentUser;
    private static String currentUserRole;

    public static void main(String[] args) {

        // Initialize files
        FileUtil.initializeFiles();

        // Initialize services
        inventoryService = new InventoryService();
        billingService = new BillingService(inventoryService);
        authenticationService = new AuthenticationService();

        // Display welcome screen
        displayWelcomeScreen();

        // User authentication
        try {
            if (authenticationService.authenticate()) {
                System.out.println("Authentication failed. Exiting...");
                return;
            }
        } catch (AuthenticationException e) {
            System.out.println("X " + e.getMessage());
            return;
        }

        // Main menu loop
        boolean running = true;
        while (running) {
            displayMainMenu();
            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1":
                    productManagementMenu();
                    break;
                case "2":
                    inventoryManagementMenu();
                    break;
                case "3":
                    billingMenu();
                    break;
                case "4":
                    customerManagementMenu();
                    break;
                case "5":
                    reportMenu();
                    break;
                case "6":
                    running = false;
                    exitApplication();
                    break;
                default:
                    System.out.println("X Invalid choice. Please try again.");
            }
        } catch (Exception e) {
            System.out.println("X Error: " + e.getMessage());
        }
    }

    // Product Management Menu
    private static void productManagementMenu() {
        boolean back = false;

        while (!back) {
            System.out.println("\n===== PRODUCT MANAGEMENT =====");
            System.out.println("1. Add New Product");
            System.out.println("2. View All Products");
            System.out.println("3. Search Product");
            System.out.println("4. Update Product");
            System.out.println("5. Delete Product");
            System.out.println("6. Back to Main Menu");
            System.out.println("Enter your choice: ");

            String choice = scanner.nextLine().trim();

            try {
                switch (choice) {
                    case "1":
                        addNewProduct();
                        break;
                    case "2":
                        inventoryService.viewAllProducts();
                        break;
                    case "3":
                        searchProduct();
                        break;
                    case "4":
                        updateProduct();
                        break;
                    case "5":
                        deleteProduct();
                        break;
                    case "6":
                        back = true;
                        break;
                    default:
                        System.out.println("X Invalid choice.");
                }
            } catch (Exception e) {
                System.out.println("X Error: " + e.getMessage());
            }
        }
    }

    // Add New Product
    private static void addNewProduct() {
        try {
            System.out.println("\n--- Add New Product ---");

            Product product = inventoryService.generateNewProduct();
            System.out.println("Product ID (Auto-generated): " + product.getId());

            System.out.println("Product Name: ");
            String name = scanner.nextLine().trim();
            ValidationUtil.validateNotEmpty(name, "Product Name");

            System.out.println("Category: ");
            String category = scanner.nextLine().trim();
            ValidationUtil.validateNotEmpty(category, "Category");

            System.out.println("Price: ");
            String priceStr = scanner.nextLine().trim();
            double price = ValidationUtil.validatePositiveDouble(priceStr, "Price");

            System.out.println("Stock Quantity: ");
            String stockStr = scanner.nextLine().trim();
            int stock = ValidationUtil.validatePositiveInteger(stockStr, "Stock Quantity");

            System.out.println("Description: ");
            String description = scanner.nextLine().trim();

            Product product = new Product(product.getId(), name, category, price, stock, description);
            inventoryService.addProduct(product);

        } catch (InvalidInputException e) {
            System.out.println("X " + e.getMessage());
        }
    }
}
```

Java: Ready

Search

File Edit Selection View Go Run Terminal Help

J MainApp.java • RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

```
15 public class MainApp {
16     // Product Management Menu
17     private static void productManagementMenu() {
18         boolean back = false;
19
20         while (!back) {
21             System.out.println("\n===== PRODUCT MANAGEMENT =====");
22             System.out.println("1. Add New Product");
23             System.out.println("2. View All Products");
24             System.out.println("3. Search Product");
25             System.out.println("4. Update Product");
26             System.out.println("5. Delete Product");
27             System.out.println("6. Back to Main Menu");
28             System.out.println("Enter your choice: ");
29
30             String choice = scanner.nextLine().trim();
31
32             try {
33                 switch (choice) {
34                     case "1":
35                         addNewProduct();
36                     case "2":
37                         inventoryService.viewAllProducts();
38                     case "3":
39                         searchProduct();
40                     case "4":
41                         updateProduct();
42                     case "5":
43                         deleteProduct();
44                     case "6":
45                         back = true;
46                     default:
47                         System.out.println("X Invalid choice.");
48                 }
49             } catch (Exception e) {
50                 System.out.println("X Error: " + e.getMessage());
51             }
52         }
53     }
54
55     // Add New Product
56     private static void addNewProduct() {
57         try {
58             System.out.println("\n--- Add New Product ---");
59
60             Product product = inventoryService.generateNewProduct();
61             System.out.println("Product ID (Auto-generated): " + product.getId());
62
63             System.out.println("Product Name: ");
64             String name = scanner.nextLine().trim();
65             ValidationUtil.validateNotEmpty(name, "Product Name");
66
67             System.out.println("Category: ");
68             String category = scanner.nextLine().trim();
69             ValidationUtil.validateNotEmpty(category, "Category");
70
71             System.out.println("Price: ");
72             String priceStr = scanner.nextLine().trim();
73             double price = ValidationUtil.validatePositiveDouble(priceStr, "Price");
74
75             System.out.println("Stock Quantity: ");
76             String stockStr = scanner.nextLine().trim();
77             int stock = ValidationUtil.validatePositiveInteger(stockStr, "Stock Quantity");
78
79             System.out.println("Description: ");
80             String description = scanner.nextLine().trim();
81
82             Product product = new Product(product.getId(), name, category, price, stock, description);
83             inventoryService.addProduct(product);
84
85         } catch (InvalidInputException e) {
86             System.out.println("X " + e.getMessage());
87         }
88     }
89 }
```

Java: Ready

Search

File Edit Selection View Go Run Terminal Help

J MainApp.java • RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

```
15 public class MainApp {
16     // User authentication
17     private static void displayWelcomeScreen() {
18         System.out.println("\n===== WELCOME =====");
19         System.out.println("RETAIL POS SYSTEM v1.0");
20         System.out.println("Developed by Team of RetailRangers");
21         System.out.println("(Suraj Arora, Arjun Pandey, Vaibhavi Nand, Aksh Mishra)");
22     }
23
24     // User authentication
25     private static boolean authenticate() throws AuthenticationException {
26         System.out.println("===== USER LOGIN =====");
27
28         int attempts = 0;
29         int maxAttempts = 3;
30
31         while (attempts < maxAttempts) {
32             try {
33                 System.out.print("Username: ");
34                 String username = scanner.nextLine().trim();
35                 System.out.print("Password: ");
36                 String password = scanner.nextLine().trim();
37
38                 // Read users from file
39                 ArrayList<String> users = FileUtil.readFromFile(FileUtil.USER_FILE);
40
41                 for (String line : users) {
42                     String[] parts = line.split(":");
43                     if (parts.length >= 3) {
44                         if (parts[0].equals(username) && parts[1].equals(password)) {
45                             currentUser = username;
46                             currentUserRole = parts[2];
47                             System.out.println("\nLogin successful!");
48                             System.out.println("Welcome, " + username + " (" + currentUserRole + ")");
49                             Thread.sleep(1000);
50                             return true;
51                         }
52                     }
53                 }
54
55                 attempts++;
56                 System.out.println("X Invalid credentials. Attempts remaining: " + (maxAttempts - attempts));
57             } catch (IOException e) {
58                 throw new AuthenticationException("Error reading user data: " + e.getMessage());
59             } catch (InterruptedException e) {
60                 // Ignore
61             }
62
63             throw new AuthenticationException("Maximum login attempts exceeded");
64         }
65     }
66
67     // Display main menu
68     private static void displayMainMenu() {
69         System.out.println("\n===== MAIN MENU =====");
70         System.out.println("1. Product Management");
71         System.out.println("2. Inventory Management");
72         System.out.println("3. Billing System");
73         System.out.println("4. Customer Management");
74         System.out.println("5. Reports");
75         System.out.println("6. Exit");
76         System.out.println("Enter your choice: ");
77     }
78 }
```

Java: Ready

Search

File Edit Selection View Go Run Terminal Help

J MainApp.java • RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

```
15 public class MainApp {
16     // Search Product
17     private static void searchProduct() {
18         System.out.println("\nEnter search term (ID/Name/Category): ");
19         String searchTerm = scanner.nextLine().trim();
20
21         ArrayList<Product> results = inventoryService.searchProducts(searchTerm);
22
23         if (results.isEmpty()) {
24             System.out.println("No products found.");
25         } else {
26             System.out.println("\n--- Search Results ---");
27             for (Product product : results) {
28                 product.displayProduct();
29             }
30         }
31     }
32
33     // Update Product
34     private static void updateProduct() {
35         try {
36             System.out.println("\nEnter Product ID to update: ");
37             String productID = scanner.nextLine().trim();
38
39             Product existingProduct = inventoryService.getProduct(productID);
40             existingProduct.displayProduct();
41
42             System.out.println("\nEnter new details (press enter to keep current value):");
43
44             System.out.print("Product Name: " + existingProduct.getName() + " ");
45             String name = scanner.nextLine().trim();
46             if (name.isEmpty()) {
47                 existingProduct.setName(name);
48             }
49
50             System.out.print("Category: " + existingProduct.getCategory() + " ");
51             String category = scanner.nextLine().trim();
52             if (category.isEmpty()) {
53                 existingProduct.setCategory(category);
54             }
55
56             System.out.print("Price: " + existingProduct.getPrice() + " ");
57             String priceStr = scanner.nextLine().trim();
58             if (priceStr.isEmpty()) {
59                 double price = ValidationUtil.validatePositiveDouble(priceStr, "Price");
60                 existingProduct.setPrice(price);
61             }
62
63             System.out.print("Stock Quantity: " + existingProduct.getStockQuantity() + " ");
64             String stockStr = scanner.nextLine().trim();
65             if (stockStr.isEmpty()) {
66                 int stock = ValidationUtil.validatePositiveInteger(stockStr, "Stock");
67                 existingProduct.setStockQuantity(stock);
68             }
69
70             System.out.print("Description: " + existingProduct.getDescription() + " ");
71             String description = scanner.nextLine().trim();
72             if (description.isEmpty()) {
73                 existingProduct.setDescription(description);
74             }
75
76             inventoryService.updateProduct(productID, existingProduct);
77
78         } catch (ProductNotFoundException e) {
79             System.out.println("X " + e.getMessage());
80         }
81     }
82
83     // Delete Product
84     private static void deleteProduct() {
85         try {
86             System.out.println("\nEnter Product ID to delete: ");
87             String productID = scanner.nextLine().trim();
88
89             Product product = inventoryService.getProduct(productID);
90             product.displayProduct();
91
92             System.out.println("Are you sure you want to delete this product? (yes/no): ");
93             String confirm = scanner.nextLine().trim().toLowerCase();
94
95             if (confirm.equals("yes")) {
96                 inventoryService.deleteProduct(productID);
97             }
98         }
99     }
100 }
```

Java: Ready

Search

File Edit Selection View Go Run Terminal Help

← → Retail-POS-System File Edit Selection View Go Run Terminal Help

← → Retail-POS-System

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void deleteProduct() {
17         Scanner scanner = new Scanner(System.in);
18         System.out.println("X * * * *");
19     }
20 }
21
22 /**
23  * Inventory Management Menu
24  */
25 private static void inventoryManagementMenu() {
26     boolean back = false;
27
28     while (!back) {
29         System.out.println("\n--- INVENTORY MANAGEMENT ---");
30         System.out.println("1. Check Stock");
31         System.out.println("2. Add Stock");
32         System.out.println("3. Low Stock Alert");
33         System.out.println("4. Inventory Value");
34         System.out.println("5. Back to Main Menu");
35         System.out.print("Enter your choice: ");
36         String choice = scanner.nextLine().trim();
37
38         try {
39             switch (choice) {
40                 case "1":
41                     checkStock();
42                     break;
43                 case "2":
44                     addStock();
45                     break;
46                 case "3":
47                     inventoryService.displayLowStockAlert();
48                     break;
49                 case "4":
50                     displayInventoryValue();
51                     break;
52                 case "5":
53                     back = true;
54                     break;
55                 default:
56                     System.out.println("X Invalid choice.");
57             }
58         } catch (Exception e) {
59             System.out.println("X Error: " + e.getMessage());
60         }
61     }
62 }
63
64 /**
65  * Check stock for a product
66  */
67 private static void checkStock() {
68     try {
69         System.out.print("Enter Product ID: ");
70         String productId = scanner.nextLine().trim();
71
72         Product product = inventoryService.getProduct(productId);
73
74         System.out.println("\n--- Stock Information ---");
75         System.out.println("Product ID : " + product.getId());
76         System.out.println("Product Name : " + product.getName());
77         System.out.println("Available Stock : " + product.getStock());
78         System.out.println("Status : " + (product.isInStock() ? "In Stock" : "Out of Stock"));
79     } catch (ProductNotFoundException e) {
80         System.out.println("X * " + e.getMessage());
81     }
82 }
83
84 /**
85  * Add stock to a product
86  */
87 private static void addStock() {
88     try {
89         System.out.print("Enter Product ID: ");
90         String productId = scanner.nextLine().trim();
91
92         Product product = inventoryService.getProduct(productId);
93     }
94 }
95
96 main Java: Ready

```

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void addCustomer() {
17         Scanner scanner = new Scanner(System.in);
18         String phone = scanner.nextLine().trim();
19         ValidationUtil.validatePhoneNumber(phone);
20
21         System.out.print("Email: ");
22         String email = scanner.nextLine().trim();
23         ValidationUtil.validateEmail(email);
24
25         Customer customer = new Customer(customerId, name, phone, email);
26         billingService.addCustomer(customer);
27     } catch (InvalidInputException e) {
28         System.out.println("X * " + e.getMessage());
29     }
30 }
31
32 private static void searchCustomer() {
33     System.out.print("Enter search term (ID/Name/Phone): ");
34     String searchTerm = scanner.nextLine().trim();
35
36     ArrayList<Customer> results = billingService.searchCustomers(searchTerm);
37
38     if (results.isEmpty()) {
39         System.out.println("No customers found.");
40     } else {
41         System.out.println("\n--- Search Results ---");
42         for (Customer customer : results) {
43             customer.displayCustomer();
44         }
45     }
46 }
47
48 /**
49  * Reports Menu
50  */
51 private static void reportsMenu() {
52     boolean back = false;
53
54     while (!back) {
55         System.out.println("\n--- REPORTS ---");
56         System.out.println("1. Sales Report");
57         System.out.println("2. Inventory Report");
58         System.out.println("3. Customer Report");
59         System.out.println("4. Back to Main Menu");
60         System.out.print("Enter your choice: ");
61         String choice = scanner.nextLine().trim();
62
63         try {
64             switch (choice) {
65                 case "1":
66                     billingService.generateSalesReport();
67                     break;
68                 case "2":
69                     inventoryService.viewAllProducts();
70                     displayInventoryValue();
71                     break;
72                 case "3":
73                     billingService.viewAllCustomers();
74                     break;
75                 case "4":
76                     back = true;
77                     break;
78                 default:
79                     System.out.println("X Invalid choice.");
80             }
81         } catch (Exception e) {
82             System.out.println("X Error: " + e.getMessage());
83         }
84     }
85 }
86
87 /**
88  * Exit application
89  */
90 private static void exitApplication() {
91     System.out.println("\n-----");
92     System.out.println("Thank you for using Retail POS System!");
93     System.out.println("Goodbye!");
94     scanner.close();
95 }
96
97 main Java: Ready

```

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void addStock() {
17         Scanner scanner = new Scanner(System.in);
18         String productId = scanner.nextLine().trim();
19
20         Product product = inventoryService.getProduct(productId);
21         System.out.println("Current Stock: " + product.getStock());
22
23         System.out.print("Enter quantity to add: ");
24         String qtyStr = scanner.nextLine().trim();
25         int quantity = ValidationUtil.validatePositiveInteger(qtyStr, fieldName: "Quantity");
26         inventoryService.addStock(productId, quantity);
27
28         System.out.println("New Stock: " + inventoryService.getProduct(productId).getStock());
29     } catch (ProductNotFoundException | InvalidInputException e) {
30         System.out.println("X * " + e.getMessage());
31     }
32 }
33
34 /**
35  * Display total inventory value
36  */
37 private static void displayInventoryValue() {
38     double totalValue = inventoryService.getTotalInventoryValue();
39     int productCount = inventoryService.getProductCount();
40
41     System.out.println("\n--- Inventory Summary ---");
42     System.out.println("Total Products : " + productCount);
43     System.out.println("Total Inventory Value: Rs. " + String.format("%.2f", totalValue));
44 }
45
46 /**
47  * Billing Menu
48  */
49 private static void billingMenu() {
50     boolean back = false;
51
52     while (!back) {
53         System.out.println("\n--- BILLING SYSTEM ---");
54         System.out.println("1. Create New Bill");
55         System.out.println("2. View All Bills");
56         System.out.println("3. Search Bill");
57         System.out.println("4. Back to Main Menu");
58         System.out.print("Enter your choice: ");
59         String choice = scanner.nextLine().trim();
60
61         try {
62             switch (choice) {
63                 case "1":
64                     createBill();
65                     break;
66                 case "2":
67                     billingService.viewAllBills();
68                     break;
69                 case "3":
70                     searchBill();
71                     break;
72                 case "4":
73                     back = true;
74                     break;
75                 default:
76                     System.out.println("X Invalid choice.");
77             }
78         } catch (Exception e) {
79             System.out.println("X Error: " + e.getMessage());
80         }
81     }
82 }
83
84 /**
85  * Create new bill
86  */
87 private static void createBill() {
88     try {
89         System.out.print("\n--- Create New Bill ---");
90         System.out.print("Enter Customer ID: ");
91         String customerId = scanner.nextLine().trim();
92
93         while (addingItems == true) {
94             System.out.print("Enter Product ID (or 'done' to finish): ");
95             String productId = scanner.nextLine().trim();
96
97             if (productId.equalsIgnoreCase("done")) {
98                 break;
99             }
100             System.out.print("Enter Quantity: ");
101             String qtyStr = scanner.nextLine().trim();
102             int quantity = ValidationUtil.validatePositiveInteger(qtyStr, fieldName: "Quantity");
103
104             try {
105                 billingService.addToBill(productId, quantity);
106             } catch (ProductNotFoundException | InsufficientStockException e) {
107                 System.out.println("X * " + e.getMessage());
108                 System.out.print("Continue adding items? (yes/no): ");
109                 if (scanner.nextLine().trim().equalsIgnoreCase("yes")) {
110                     addingItems = false;
111                 }
112             }
113         }
114
115         if (bill.getTotalCount() > 0) {
116             System.out.print("Confirm billing? (yes/no): ");
117             String confirm = scanner.nextLine().trim().toLowerCase();
118
119             if (confirm.equals("yes")) {
120                 billingService.completeBilling(bill);
121             } else {
122                 System.out.println("Billing cancelled.");
123             }
124         } else {
125             System.out.println("No items added. Billing cancelled.");
126         }
127     } catch (InvalidInputException | ProductNotFoundException | InsufficientStockException e) {
128         System.out.println("X * " + e.getMessage());
129     }
130 }
131
132 /**
133  * Search bill
134  */
135 private static void searchBill() {
136     System.out.print("Enter search term (Bill ID/Customer ID/Name): ");
137     String searchTerm = scanner.nextLine().trim();
138
139     ArrayList<Bill> results = billingService.searchBills(searchTerm);
140
141     if (results.isEmpty()) {
142         System.out.println("No bills found.");
143     } else {
144         System.out.println("\n--- Search Results ---");
145         for (Bill bill : results) {
146             bill.displayInvoice();
147         }
148     }
149 }
150
151 /**
152  * Customer Management Menu
153  */
154 private static void customerManagementMenu() {
155     boolean back = false;
156
157     while (!back) {
158         System.out.println("\n--- CUSTOMER MANAGEMENT ---");
159         System.out.println("1. Add New Customer");
160         System.out.println("2. View All Customers");
161     }
162 }
163
164 main Java: Ready

```

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void createBill() {
17         Scanner scanner = new Scanner(System.in);
18         System.out.print("\n--- Create New Bill ---");
19
20         System.out.print("Enter Customer ID: ");
21         String customerId = scanner.nextLine().trim();
22
23         Bill bill = billingService.createBill(customerId, currentUser);
24         System.out.println("Bill created: " + bill.getId());
25
26         boolean addingItems = true;
27         while (addingItems == true) {
28             System.out.print("Enter Product ID (or 'done' to finish): ");
29             String productId = scanner.nextLine().trim();
30
31             if (productId.equalsIgnoreCase("done")) {
32                 break;
33             }
34             System.out.print("Enter Quantity: ");
35             String qtyStr = scanner.nextLine().trim();
36             int quantity = ValidationUtil.validatePositiveInteger(qtyStr, fieldName: "Quantity");
37
38             try {
39                 billingService.addToBill(productId, quantity);
40             } catch (ProductNotFoundException | InsufficientStockException e) {
41                 System.out.println("X * " + e.getMessage());
42                 System.out.print("Continue adding items? (yes/no): ");
43                 if (scanner.nextLine().trim().equalsIgnoreCase("yes")) {
44                     addingItems = false;
45                 }
46             }
47         }
48
49         if (bill.getTotalCount() > 0) {
50             System.out.print("Confirm billing? (yes/no): ");
51             String confirm = scanner.nextLine().trim().toLowerCase();
52
53             if (confirm.equals("yes")) {
54                 billingService.completeBilling(bill);
55             } else {
56                 System.out.println("Billing cancelled.");
57             }
58         } else {
59             System.out.println("No items added. Billing cancelled.");
60         }
61     } catch (InvalidInputException | ProductNotFoundException | InsufficientStockException e) {
62         System.out.println("X * " + e.getMessage());
63     }
64 }
65
66 /**
67  * Search bill
68  */
69 private static void searchBill() {
70     System.out.print("Enter search term (Bill ID/Customer ID/Name): ");
71     String searchTerm = scanner.nextLine().trim();
72
73     ArrayList<Bill> results = billingService.searchBills(searchTerm);
74
75     if (results.isEmpty()) {
76         System.out.println("No bills found.");
77     } else {
78         System.out.println("\n--- Search Results ---");
79         for (Bill bill : results) {
80             bill.displayInvoice();
81         }
82     }
83 }
84
85 /**
86  * Customer Management Menu
87  */
88 private static void customerManagementMenu() {
89     boolean back = false;
90
91     while (!back) {
92         System.out.println("\n--- CUSTOMER MANAGEMENT ---");
93         System.out.println("1. Add New Customer");
94         System.out.println("2. View All Customers");
95     }
96 }
97
98 main Java: Ready

```